

03 - Routing, Templates e Base

1. Estruturando a aplicação como um sistema profissional

Até agora os foi visto:

- O que é uma aplicação
- 2.Desenvolvimento de Aplicações...
- Diferença entre backend e frontend
 - O que é servidor, rota e endpoint
- 3.Desenvolvimento de Aplicações...
- Criar rotas simples que retornam texto ou JSON

Mas existe um problema.



Um app que retorna:
return "Olá mundo"
não é uma aplicação real.

Hoje vamos aprender a transformar:
Um conjunto de rotas soltas
em
Uma aplicação estruturada, modular e escalável.

Hoje é **estrutura e arquitetura inicial**:

- Organizar rotas
- Criar páginas HTML reais
- Criar layout reutilizável
- Tratar erros corretamente
- Preparar o projeto para crescer

2. Porque aplicações reais crescem

Um sistema pequeno	5 rotas
Um sistema médio	50 rotas
Um sistema grande	300+ rotas

Respostas:

- Arquivo gigante
- Difícil manter
- Conflito entre desenvolvedores
- Testes mais difíceis

Sem modularização:

- Código vira caos
- Conflitos aumentam
- Manutenção fica lenta
- Testes ficam difíceis

3. Exemplo: 50 rotas em um único app.py (caótico)

```
from flask import Flask, request, render_template

app = Flask(__name__)

# ===== 1) PÁGINAS PÚBLICAS (10 rotas) =====
@app.get("/")
def home(): return render_template("home.html")

@app.get("/sobre")
def sobre(): return render_template("sobre.html")

@app.get("/contato")
def contato(): return render_template("contato.html")

@app.post("/contato")
def contato_enviar(): return {"ok": True}
...
```

- **Quero destacar no menu qual página está ativa**
 - normalmente você usa `request.endpoint` e compara
 - alguém cria uma nova rota e esquece de atualizar a lógica do “ativo”
- **Agora admin precisa de proteção, público não**
 - cria um decorator e tem que lembrar de colocar em todas
 - alguém cria uma rota nova em `/admin/relatórios`, e esquece o decorator

4. O que são Blueprints no Flask?

Blueprint é um mecanismo do Flask para **organizar a aplicação em módulos independentes**. Em vez de colocar todas as rotas em um único arquivo (por exemplo, `app.py`), você divide o sistema por domínios.

- No Django, o equivalente a um *blueprint* é um **app Django**.
- FastAPI não chama de *blueprints*, mas usa `APIRouter`, que cumpre exatamente o mesmo papel.

Em vez de:

- `app.py` com 80 rotas

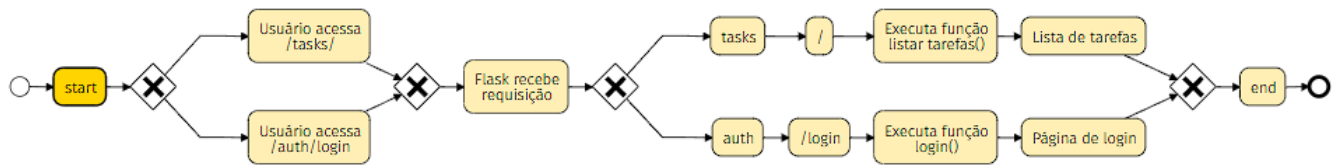
Você faz:

- módulo de tarefas
- módulo de autenticação
- módulo administrativo

4.1. Exemplo – Exemplo_03

Preciso desta estrutura:

```
Exemplo_03/
├── run.py
├── src/
│   ├── __init__.py
│   ├── microtask/
│   │   ├── __init__.py
│   │   └── blueprints/
│   │       ├── __init__.py
│   │       ├── auth/
│   │       │   ├── __init__.py
│   │       │   └── routes.py
│   │       └── tasks/
│   │           ├── __init__.py
│   │           └── routes.py
```



Quando você cria um Blueprint, ele é apenas um **módulo isolado** contendo rotas, funções, middlewares etc. Mas ele **não faz parte da aplicação Flask** até que você o registre, usando este método:

```
app.register_blueprint(tasks_bp)
```

Com isso ele **importa as rotas do Blueprint para dentro do app**. Antes disso, o Flask nem sabe que essas rotas existem.

src/ Diretório do código-fonte • Separa código da raiz. • Evita misturar execução com lógica. • Estrutura usada em projetos reais. Organização profissional.	src/microtask/ Aplicação principal • Representa o sistema. • Contém a configuração do app. • Registra os módulos (Blueprints). Aqui nasce a aplicação Flask.
---	---

• src/microtasks/blueprints/auth/routes.py

Rotas de autenticação, por exemplo:

- GET /login
- POST /login
- Logout
- Lógica de sessão

Aqui vivem os endpoints.

```
from . import bp

@bp.route("/login")
def login():
    return "Página de login"
```

• src/microtasks/blueprints/auth/__init__.py

Criação do Blueprint de autenticação

- Cria bp = Blueprint("auth", __name__)
- Importa routes.py
- Registra as rotas dentro do módulo.

Sem isso, o módulo não existe para o Flask.

```
from flask import Blueprint

bp = Blueprint("auth", __name__)

from . import routes
```

• src/microtasks/blueprints/tasks/routes.py

Rotas de tarefas, por exemplo:

- Listar tarefas
- Criar tarefa
- Marcar como concluída
- Retornar HTML ou JSON

Aqui está a regra de negócio.

```
from . import bp

@bp.route("/")
def listar_tarefas():
    return "Lista de tarefas"
```

- **src/microtasks/blueprints/tasks/__init__.py**

Criação do Blueprint de tarefas

- Cria bp = Blueprint("tasks", __name__)
- Importa routes.py

Conecta as rotas ao módulo.

```
from flask import Blueprint

bp = Blueprint("tasks", __name__)

from . import routes
```

- **src/microtasks/__init__.py**

Fábrica da aplicação (create_app)

- Cria o objeto Flask.
- Registra Blueprints.
- Configura o sistema.

É o “cérebro inicial”.

```
from flask import Flask

def create_app():
    app = Flask(__name__)

    # Importa os blueprints
    from .blueprints.auth import bp as auth_bp
    from .blueprints.tasks import bp as tasks_bp

    # Registra os blueprints
    app.register_blueprint(auth_bp, url_prefix="/auth")
    app.register_blueprint(tasks_bp, url_prefix="/tasks")

    return app
```

- **run.py**

Ponto de entrada da aplicação

- Cria a aplicação Flask.
- Chama create_app().
- Inicia o servidor.
- Só existe para rodar o sistema.

Ele não contém regra de negócio.

```
from src.microtask import create_app
app = create_app()
if __name__ == "__main__":
    app.run(debug=True)
```

5. Templates — Separando Lógica de Apresentação

Até agora: Backend estava retornando string.

Mas aplicações reais retornam HTML estruturado.

5.1. O que é Template?

Template é um arquivo HTML que pode receber dados do backend.

Flask usa Jinja2, que 2 é o **motor de templates** mais usado no ecossistema Python — especialmente em frameworks web como **Flask**.

- Você escreve arquivos HTML (ou texto de qualquer tipo)
- Dentro deles, usa **expressões, variáveis e estruturas de controle**
- O Python envia dados para o template
- O Jinja2 monta o HTML final

Ele funciona como uma camada entre o backend e o HTML.

5.2. Por que Jinja2 existe?

- Sem Jinja2, você teria que montar HTML com Python assim:
 - `html = "<h1>Olá, " + nome + "</h1>"`
 - Isso é horrível de manter.
- Com Jinja2, você faz:
 - `<h1>Olá, {{ nome }}</h1>`
 - Muito mais limpo.

5.3. Conceitos principais do Jinja2

5.3.1. Variáveis

```
<p>Usuário: {{ usuario.nome }}</p>
```

5.3.2. Estruturas de controle

- **Condicionais (HTML)**

```
{% if usuario.admin %}  
<p>Bem-vindo, administrador!</p>  
{% endif %}
```

- **Loops (HTML)**

```
<ul>  
{% for item in lista %}  
<li>{{ item }}</li>  
{% endfor %}  
</ul>
```

- **Filtros (HTML)**

```
<p>{{ nome|upper }}</p>  
○ Filtros comuns:

- upper
- lower
- title
- safe
- length
- replace

```

- **{% block content %} {% endblock %}**

Usado pelo Flask, para definir blocos substituíveis dentro de um template base. Ele faz parte do mecanismo de herança de templates, que permite criar um layout padrão (como um “template pai”) e depois sobrescrever apenas partes específicas em páginas filhas.

- {% block content %}: marca o início de um bloco nomeado content.
- {% endblock %}: marca o fim desse bloco.

Tudo que estiver dentro desse bloco pode ser substituído por templates que herdarem esse arquivo.

- **{% block title %} {% endblock %}**

Usado pelo Flask, e serve para **definir uma área substituível** dentro de um template base.

5.3.3. Herança de templates - um dos recursos mais poderosos

Isso evita repetição de HTML.

- **Template base:**

```
<!DOCTYPE html>
<html>
<head>
<title>{% block title %}{% endblock %}</title>
</head>
<body>
{% block content %}{% endblock %}
</body>
</html>
```

- **Template filho:**

```
{% extends "base.html" %}
{% block title %}Página Inicial{% endblock %}
{% block content %}
<h1>Bem-vindo!</h1>
{% endblock %}
```

5.3.4. Como ele se encaixa no Flask

- **Aplicativo**

```
from flask import Flask, render_template
app = Flask(__name__)
@app.route("/")
def home():
    return render_template("index.html", nome="Salvador")
```

- **Template (index.html):**

```
<h1>Olá, {{ nome }}!</h1>
```

5.3.5. Segurança

Jinja2 automaticamente **escapa HTML perigoso**, evitando ataques XSS.

- **Python**

```
nome = "<script>alert('hack')</script>"
```

- **No template:**

```
{{ nome }}
```

- **Resultado exibido:**

`<script>alert('hack')</script>`
Isso protege sua aplicação.

5.4. Exemplo

```
src/  
└─ microtask/  
    ├── __init__.py  
    ├── blueprints/  
    │   ├── tasks/  
    │   │   ├── __init__.py  
    │   │   └─ routes.py  
    │   └─ auth/  
    │       ├── __init__.py  
    │       └─ routes.py  
    └─ templates/  
        └─ tasks/  
            └─ listar_tarefas.html
```

- **ALTERAR:** `src/microtasks/blueprints/tasks/routes.py`

```
from flask import render_template  
from . import bp  
  
@bp.route("/")  
def listar_tarefas():  
    # Dados simples vindos do backend (só pra mostrar o fluxo)  
    tarefas = ["Estudar Flask", "Fazer exercícios", "Tomar café (forte)"]  
    return render_template("tasks/listar_tarefas.html", tarefas=tarefas)
```

- **CRIAR:** `src/microtask/templates/tasks/listar_tarefas.html`

```
<!doctype html>  
<html lang="pt-br">  
<head>  
<meta charset="utf-8" />  
<title>Lista de tarefas</title>  
</head>  
<body>  
<h1>Lista de tarefas</h1>  
{% if tarefas %}  
<ul>  
{% for t in tarefas %}  
<li>{{ t }}</li>  
{% endfor %}  
</ul>  
{% else %}  
<p>Nenhuma tarefa cadastrada.</p>  
{% endif %}  
</body>  
</html>
```

Então é só acessar:

- <http://127.0.0.1:5000/tasks/>

Atenção:

- antes: rota devolvia string
- agora: rota devolve **HTML gerado** a partir de um template

6. Base Layout — Layout Reutilizável

Se cada página tiver:

- Cabeçalho repetido
- Menu repetido
- CSS repetido

Você vai copiar e colar código.

6.1. Solução: base

Esse arquivo é um **template base**, ou seja, um *layout mestre*. Ele define a estrutura comum que todas as páginas do site devem compartilhar:

- `<html>`, `<head>`, `<body>`
- `<header>`
- `<main>`
- links de CSS
- blocos que podem ser substituídos

Ele funciona como um molde.

6.2. O que permite que outras páginas herdem esse template?

O mecanismo de herança de templates do Jinja2, ativado pelo comando:

```
{% extends "base.html" %}
```

Quando um template usa `extends`, o Jinja2:

1. Carrega o `base.html`
2. Procura os blocos definidos nele (`block title`, `block content`)
3. Substitui esses blocos pelo conteúdo do template filho
4. Renderiza tudo como um único HTML final

É literalmente herança — como em orientação a objetos, mas para HTML.

6.3. Exemplo

```
templates/
├─ base.html
└─ tasks/
   └─ listar_tarefas.html
```

- **ALTERE:** `src/microtask/templates/tasks/listar_tarefas.html`

```
{% extends "base.html" %}
{% block title %}Tarefas{% endblock %}
{% block content %}
<h1>Lista de tarefas</h1>
{% if tarefas %}
<ul>
{% for t in tarefas %}
<li>{{ t }}</li>
{% endfor %}
</ul>
{% else %}
<p>Nenhuma tarefa cadastrada.</p>
```

```
{% endif %}  
{% endblock %}
```

- **CRIAR: src/microtask/templates/base.html**

```
<!doctype html>  
<html lang="pt-br">  
  <head>  
    <meta charset="utf-8" />  
    <title>{% block title %}MicroTask{% endblock %}</title>  
  </head>  
  <body>  
    <header>  
      <h2>MicroTask</h2>  
      <hr />  
    </header>  
  
    <main>  
      {% block content %}{% endblock %}  
    </main>  
    <footer>  
      <hr />  
      <small>Salvador Melo</small>  
    </footer>  
  </body>  
</html>
```

Em resumo:

- Sem base, a gente repetia HTML em toda página.
- Com base, a estrutura fica em base.html.
- A página filha usa extends e só preenche os blocos.
- O Flask só injeta dados; quem monta o HTML final é o Jinja2.

7. Exercício - CampusQuest

Imagine que você foi contratado para desenvolver a **primeira versão de um mini portal para estudantes do campus**. Nada complexo. Nada de banco de dados. Nada de login.

A coordenação quer algo extremamente simples para começar:

- Uma página inicial (hub)
- Uma página explicando o projeto
- Uma lista de “missões” ou tarefas acadêmicas
- Um espaço para avisos
- Uma rota personalizada que cumprimente o aluno pelo nome
- Uma página de erro amigável

Ou seja:

Não é um sistema completo.

É a **primeira versão organizada de um produto real**.

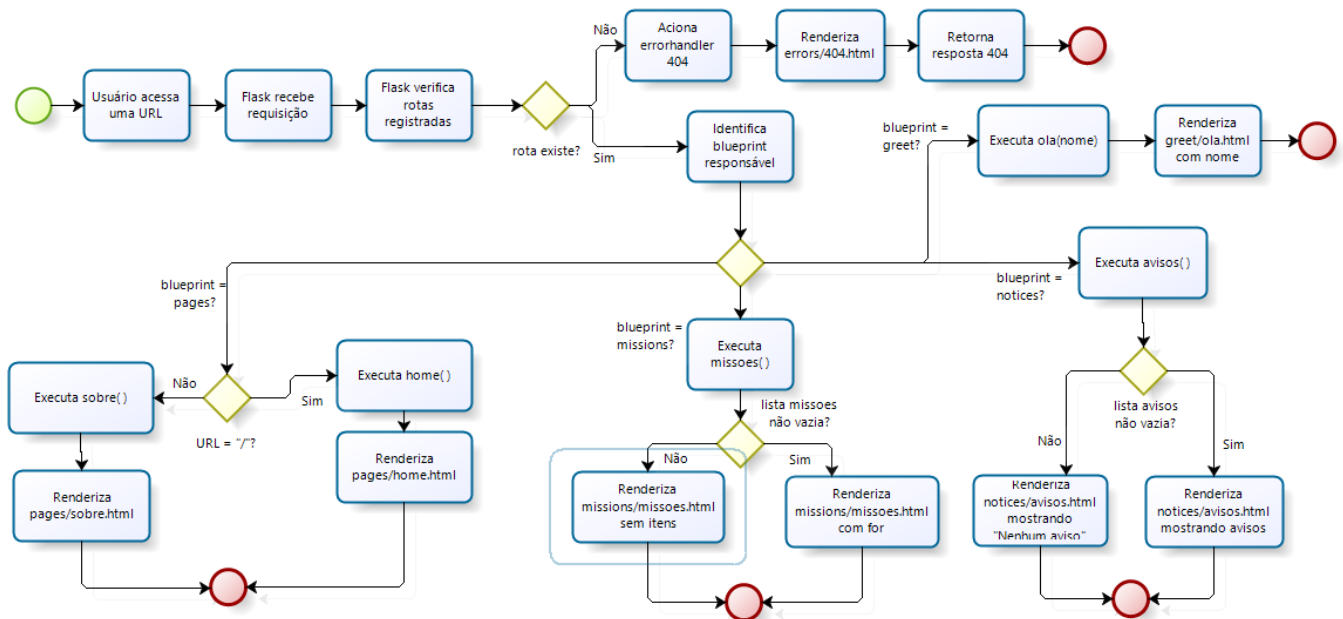
Neste projeto, vocês vão praticar:

- Organização do projeto em módulos (Blueprints)
- Criação de rotas
- Uso de templates com herança (base.html)

- Passagem de dados simples do backend para o HTML
- Uso de for e if no Jinja2
- Tratamento básico de erro (404)

Nada além disso.

Se aparecer outra técnica, é porque alguém está inventando moda.



7.1. Preparação: criar as pastas e arquivos vazios

- Objetivo: deixar o “esqueleto” pronto antes de escrever código, pra não se perder.
- Crie a pasta do projeto campusquest/.
- Dentro dela, crie:
 - run.py
 - src/microtask/__init__.py
- Crie as pastas de blueprints:
 - src/microtask/blueprints/pages/
 - src/microtask/blueprints/missions/
 - src/microtask/blueprints/notices/
 - src/microtask/blueprints/greet/
- Dentro de cada uma dessas pastas, crie:
 - __init__.py
 - routes.py
- Crie a pasta de templates:
 - src/microtask/templates/
- Dentro de templates/, crie:
 - base.html
 - pasta pages/ com home.html e sobre.html
 - pasta missions/ com missoes.html
 - pasta notices/ com avisos.html
 - pasta greet/ com ola.html
 - pasta errors/ com 404.html

Checagem rápida: quando você olhar no Explorer do VS Code, a estrutura deve parecer com um “esqueleto organizado”, mesmo com arquivos ainda vazios.

7.2. Criar o layout base (base.html)

- Objetivo: toda página vai “herdar” esse layout. Assim você não repete menu e estrutura.
- Abra o arquivo:
 - `src/microtask/templates/base.html`
- Cole um HTML simples com:
 - `<title>` usando um bloco `{% block title %}`
 - um menu com links (`/`, `/missoes`, `/avisos`, `/sobre`)
 - um bloco `{% block content %}` para cada página colocar seu conteúdo
- Salve o arquivo.

Por que fazemos isso?

Porque se amanhã você quiser mudar o menu, você muda um arquivo só.

7.3. Criar o “coração” do app: `create_app()`

- Objetivo: criar o app Flask e preparar o lugar onde você vai registrar os módulos (blueprints).
- Abra:
 - `src/microtask/__init__.py`
- Escreva o `create_app()`:
 - cria `app = Flask(__name__)`
 - (por enquanto) retorna `app`
- Salve.

Checagem rápida: não vamos rodar ainda. Aqui só estamos montando a base.

7.4. Rodar o app “vazio” (só pra testar que o Flask sobe)

- Objetivo: validar que o projeto está rodando antes de adicionar blueprints.
- Abra `run.py`
- Escreva:
 - `import create_app`
 - `cria app = create_app()`
 - `app.run(debug=True)`
- No terminal, rode:
 - `python run.py`

O que você deve ver:

O terminal mostra que o Flask subiu.

Se você abrir `http://127.0.0.1:5000/`, vai dar 404 (normal, não existe rota ainda).

Esse 404 é bom sinal: significa que o servidor está rodando, só falta criar rotas.

7.5. Blueprint “pages”: criar o módulo e a rota Home

7.5.1. Criar o blueprint

- Objetivo: criar o primeiro módulo (blueprint) e uma rota real.
- Abra:
 - `src/microtask/blueprints/pages/__init__.py`
- Crie:
 - `bp = Blueprint("pages", __name__)`

- No final do arquivo, importe as rotas:
 - `from . import routes`

Por que importar routes?

Porque o Flask só “enxerga” as rotas quando o arquivo `routes.py` é carregado.

7.5.2. Criar a rota /

- Abra:
 - `src/microtask/blueprints/pages/routes.py`
- Importe:
 - `render_template`
 - `bp`
- Crie a rota:
 - `@bp.get("/")`
 - função `home()` que retorna `render_template("pages/home.html")`

7.5.3. Criar o template home.html

- Abra:
 - `src/microtask/templates/pages/home.html`
- Faça o template:
 - `{% extends "base.html" %}`
 - `define {% block title %}`
 - `define {% block content %}`

7.5.4. Registrar o blueprint no app

- Volte no:
 - `src/microtask/__init__.py`
- Dentro do `create_app()`, importe e registre:
 - `from .blueprints.pages import bp as pages_bp`
 - `app.register_blueprint(pages_bp)`

7.5.5. Testar

Pare o servidor (CTRL + C) e rode de novo `python run.py`
Acesse:

`http://127.0.0.1:5000/`

O que você deve ver:

A Home aparece com o menu do `base.html`.

7.6. Adicionar a rota /sobre no mesmo blueprint

- Objetivo: criar uma segunda rota “no mesmo módulo”.
- No arquivo:
 - `src/microtask/blueprints/pages/routes.py`
- Crie:
 - `@bp.get("/sobre")`
 - função `sobre()` retornando `render_template("pages/sobre.html")`
- Crie o template:
 - `src/microtask/templates/pages/sobre.html` herdando `base.html`

Teste acessando:

<http://127.0.0.1:5000/sobre>

7.7. Blueprint “missions”: rota com lista (loop no template)

- Objetivo: o backend envia uma lista simples para o template.
- Crie o blueprint em:
 - `src/microtask/blueprints/missions/__init__.py`
- Crie a rota em:
 - `src/microtask/blueprints/missions/routes.py`
 - `@bp.get("/missoes")`
 - cria uma lista `missoes = [...]`
 - retorna `render_template("missions/missoes.html", missoes=missoes)`
- Crie o template:
 - `src/microtask/templates/missions/missoes.html`
 - use `{% for m in missoes %}` para listar
- Registre no `create_app()`:
 - importar `missions_bp`
 - `app.register_blueprint(missions_bp)`

Teste:

<http://127.0.0.1:5000/missoes>

7.8. Blueprint “notices”: condicional (if/else)

- Objetivo: lidar com lista vazia sem quebrar a página.
- Crie blueprint `notices` (`__init__.py` + `routes.py`)
- Rota:
 - `@bp.get("/avisos")`
 - `avisos = []` (vazio)
 - renderiza `notices/avisos.html`
- Template `avisos.html`:
 - `{% if avisos %}` mostra lista
 - `{% else %}` mostra “Nenhum aviso no momento.”
- Registre blueprint no `create_app()`

Teste:

<http://127.0.0.1:5000/avisos> (deve mostrar “Nenhum aviso...”)

Depois, você pode trocar pra:

```
avisos = ["Prova sexta", "Entrega EPIC segunda"]
```

só pra ver o outro caminho do if.

7.9. Etapa 8 — Blueprint “greet”: rota com parâmetro

- Objetivo: reforçar que a rota pode receber dado na URL.
- Crie blueprint `greet`
- Rota:
 - `@bp.get("/ola/<nome>")`

- recebe nome na função
- envia para o template: `render_template("greet/ola.html", nome=nome)`
- Template:
- mostra `{{ nome }}`

Teste:

`http://127.0.0.1:5000/ola/Amanda`

`http://127.0.0.1:5000/ola/Joao`

7.10. Etapa 9 — Página 404 customizada

- Objetivo: quando o aluno digitar URL errada, não ver a página padrão.
- Crie o template:
- `src/microtask/templates/errors/404.html`
- herdando `base.html`
- No `create_app()` (em `src/microtask/__init__.py`):
- crie o handler:
- `@app.errorhandler(404)`
- retorna `render_template("errors/404.html"), 404`

Teste:

Acesse algo inexistente, tipo:

`http://127.0.0.1:5000/qualquer-coisa`

Deve aparecer sua página de erro, com menu e link pra home.

Mini-checklist final

- `/` abre Home com menu
- `/sobre` abre Sobre
- `/missoes` lista missões
- `/avisos` mostra “Nenhum aviso” (ou lista, se preencher)
- `/ola/QualquerNome` mostra saudação
- rota inexistente mostra seu 404